Day 90 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 13, Day 7 reference

Watch: The spelled-out intro to language modeling: building makemore (Andrej Karpathy) | Transformers, the tech behind LLMs, Deep Learning Chapter 5 (3Blue1Brown)

`day90_tiny_gpt.py`, segment by segment, in the order `main()` uses things. Section 1 is Week 13's six exercise scripts' functions, copied verbatim and explained in Days 84–89's guides; Week 12's parts are imported from Day 83's build. This reference covers what is new. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: T = 64 context positions · P = every weight, in nested dictionaries · w12 = Day 83's build, imported · bits per character = nats / ln 2 / characters

Segment 0 — Setup

Importing Week 12, and the settings

CODE

```
sys.path.insert(0, os.path.join(HERE, "..", "week-12-the-transformer"))
with contextlib.redirect_stdout(io.StringIO()):
    import day83_tiny_transformer as w12
np.random.seed(42)
CORPUS = os.path.join(HERE, "corpora", "tinyshakespeare.txt")
T, B = 64, 32
D, HEADS, F, LAYERS = 64, 4, 256, 2
LR, STEPS = 3e-3, 4000
CHECK_EVERY = 500
SEEDS = [0, 1]
MERGES = 200
```

- `import day83_tiny_transformer as w12` — Week 12's parts by name: w12.attention, w12.layer_norm, w12.encoder_block_backward.
- `D, HEADS, F, LAYERS = 64, 4, 256, 2` — tuple unpacking: four settings in one line.

Segment 1 — The parts

How Week 13's parts are copied in

CODE

```
# ---- from day85_exercise.py, verbatim ----
def embed(E, ids):
    return E[ids]
def embed_backward(dX, E, ids):
    dE = np.zeros_like(E)
    np.add.at(dE, ids, dX)
    return dE
```

- `# ---- from day85_exercise.py, verbatim ----` — a banner above each group names the exercise script it comes from.
- `embed(E, ids)` — Day 85's lookup; Day 90 calls it twice, for tokens and for positions.

Segment 2 — The model

gpt_block: Day 78's block with the causal mask

MATH

$$score_{ij} = q_i \cdot k_j / \sqrt{d_k} \text{ if } j \le i, -\infty \text{ otherwise;} \quad weights = softmax \text{ over } j$$

CODE

```
def gpt_block(x, P, h):
    m, n, _ = x.shape
    km = np.broadcast_to(w12.causal_mask(n), (m * h, n, n))
    Q, K, V = (w12.split(x @ P[w], h) for w in ("Wq", "Wk", "Wv"))
    Zh, A = w12.attention(Q, K, V, km)
    Z = w12.merge(Zh, h)
    h1, n1 = w12.layer_norm(x + Z @ P["Wo"], P["g1"], P["b1"])
    u = h1 @ P["W1"] + P["c1"]; r = np.maximum(u, 0)
    y, n2 = w12.layer_norm(h1 + r @ P["W2"] + P["c2"], P["g2"], P["b2"])
    return y, (x, Q, K, V, A, Z, h1, n1, u, r, n2)
```

- `np.broadcast_to(w12.causal_mask(n), (m * h, n, n))` — one (n, n) lower triangle shown as m × h copies without copying memory.
- `(w12.split(x @ P[w], h) for w in ("Wq", "Wk", "Wv"))` — a generator unpacked into Q, K, V: three projections, each cut into heads.
- `return y, (x, Q, K, V, A, Z, ...)` — exactly Day 78's cache layout, which is why Day 78's backward pass works unchanged.

init_gpt, gpt_forward, gpt_loss_grads

MATH

$x = E[ids] + Pos[0..n-1]; \quad h = blocks(x); \quad logits = h\,W_{out} + b_{out}; \quad loss = mean\ -log\ softmax(logits)[next\ token]$

CODE

```
def init_gpt(rng, V, d=D, ff=F, layers=LAYERS, t=T):
    s = lambda a, b: rng.randn(a, b) / np.sqrt(a)
    blk = lambda: dict(Wq=s(d, d), Wk=s(d, d), Wv=s(d, d), Wo=s(d, d), W1=s(d, ff),
        c1=np.zeros(ff), W2=s(ff, d),
                       c2=np.zeros(d), g1=np.ones(d), b1=np.zeros(d), g2=np.ones(d),
                           b2=np.zeros(d))
    return dict(E=rng.randn(V, d) * 0.1, pos=rng.randn(t, d) * 0.1, blocks={str(k): blk() for k in
        range(layers)},
                Wout=s(d, V) * 0.1, bout=np.zeros(V))
def gpt_forward(ids, P, h=HEADS):
    n = ids.shape[1]
    x = embed(P["E"], ids) + embed(P["pos"], np.arange(n))
    caches = []
    for k in sorted(P["blocks"]):
        x, c = gpt_block(x, P["blocks"][k], h); caches.append(c)
    return output_layer(x, P["Wout"], P["bout"]), (ids, x, caches)
def gpt_loss_grads(ids, y, P, h=HEADS):
    logits, (ids, x, caches) = gpt_forward(ids, P, h)
    loss, dl = softmax_xent(logits, y)
    dx, dW, db = output_layer_backward(dl, x, P["Wout"])
    G = dict(Wout=dW, bout=db, blocks={})
    for k, c in reversed(list(zip(sorted(P["blocks"]), caches))):
        dx, G["blocks"][k] = w12.encoder_block_backward(dx, P["blocks"][k], h, c)
    G["E"] = embed_backward(dx, P["E"], ids)
    G["pos"] = embed_backward(dx, P["pos"], np.broadcast_to(np.arange(ids.shape[1]), ids.shape))
    return loss, G
```

- `blocks={str(k): blk() for k in range(layers)}` — a dict of blocks keyed '0', '1': Day 82's leaves() then names them blocks.0.Wq, ...
- `for k in sorted(P["blocks"])` — the blocks in order, bottom to top.
- `reversed(list(zip(sorted(P["blocks"]), caches)))` — top to bottom for the backward pass; reversed needs a list, so zip is turned into one.
- `np.broadcast_to(np.arange(n), ids.shape)` — position ids for every sequence, so Day 85's embed_backward adds up each position's gradient.

adam_step: Day 37's update

MATH

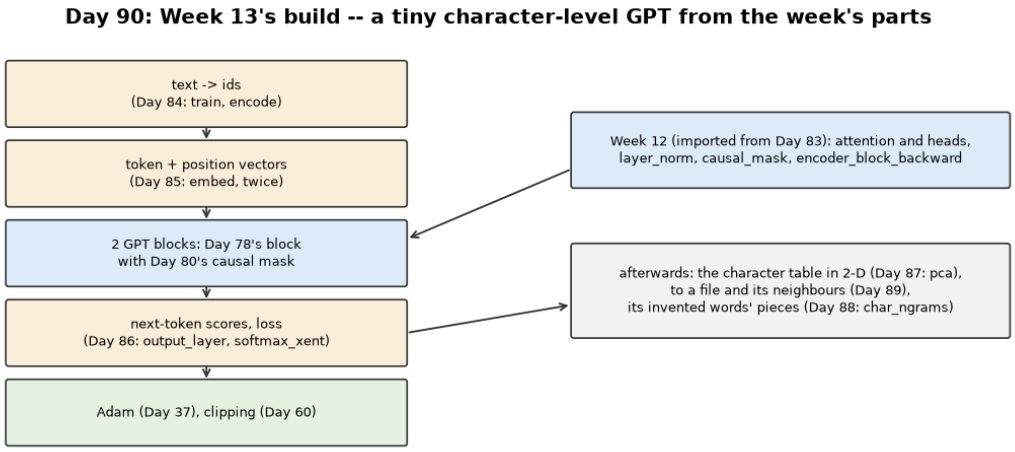
$$m \leftarrow 0.9\,m + 0.1\,g;\quad v \leftarrow 0.999\,v + 0.001\,g^2;\quad w \leftarrow w - lr \cdot (m / (1 - 0.9^t)) / (\sqrt{(v / (1 - 0.999^t))} + 10^{-8})$$

CODE

```
def adam_step(P, G, state, lr, clip=1.0):
    g = dict(w12.leaves(G))
    scale = min(1.0, clip / np.sqrt(sum(np.sum(a ** 2) for a in g.values())))
    state["t"] += 1; t = state["t"]
    for k, A in w12.leaves(P):
        m = state["m"].setdefault(k, np.zeros_like(A)); v = state["v"].setdefault(k,
            np.zeros_like(A))
        gk = g[k] * scale
        m[:] = 0.9 * m + 0.1 * gk
        v[:] = 0.999 * v + 0.001 * gk ** 2
        A -= lr * (m / (1 - 0.9 ** t)) / (np.sqrt(v / (1 - 0.999 ** t)) + 1e-8)
```

- `state["m"].setdefault(k, np.zeros_like(A))` — the moment array for weight k, created at zero the first time it is asked for.
- `m[:] = 0.9 * m + 0.1 * gk` — [:] writes into the existing array, so the one stored in state changes.
- `A -= ...` — in-place update of the weight inside P.

SYSTEM DIAGRAM



Week 13's build: the week's parts, Week 12's block, one trained model.

Segment 3 — Data, training, baselines, sampling

windows, bits_per_char, train_gpt

MATH

$$\textit{bits per character} = \Sigma\, -\ln p(\textit{token}) / \ln 2 / (\textit{characters the tokens spell})$$

CODE

```
def windows(ids):
    n = (len(ids) - 1) // T
    return ids[:n * T].reshape(n, T), ids[1:n * T + 1].reshape(n, T)
def bits_per_char(P, vx, vy, vocab):
    nats = 0.0
    for s in range(0, len(vx), 64):
```

```
        lg, _ = gpt_forward(vx[s:s + 64], P)
        nats += softmax_xent(lg, vy[s:s + 64])[0] * vy[s:s + 64].size
    chars = sum(len(vocab[i]) for i in vy.ravel())
    return nats / chars / np.log(2)
def log_softmax(lg):
    z = lg - lg.max(axis=-1, keepdims=True)
    return z - np.log(np.exp(z).sum(axis=-1, keepdims=True))
def position_bits(P, vx, vy):
    tot = np.zeros(vx.shape[1])
    for s in range(0, len(vx), 64):
        logp = log_softmax(gpt_forward(vx[s:s + 64], P)[0])
        tot -= np.take_along_axis(logp, vy[s:s + 64][..., None], axis=-1)[..., 0].sum(axis=0) /
            np.log(2)
    return tot / len(vx)
def context_bits(P, ids, vocab):
    h = T // 2
    starts = np.arange(0, len(ids) - T - 1, h)
    nats, chars = 0.0, 0
    for s in range(0, len(starts), 64):
        st = starts[s:s + 64]
        x = np.stack([ids[i:i + T] for i in st]); y = np.stack([ids[i + 1:i + T + 1] for i in st])
        logp = log_softmax(gpt_forward(x, P)[0][:, h:])
        nats -= np.take_along_axis(logp, y[:, h:, None], axis=-1).sum()
        chars += sum(len(vocab[i]) for i in y[:, h:].ravel())
    return nats / chars / np.log(2)
def train_gpt(tr, vx, vy, vocab, seed, steps=STEPS):
    rng = np.random.RandomState(seed)
    P = init_gpt(rng, len(vocab)); state = {"t": 0, "m": {}, "v": {}}
    cpt = np.array([len(v) for v in vocab])[tr].mean()
    curve, recent, busy = [(0, np.nan, bits_per_char(P, vx, vy, vocab))], [], 0.0
    for step in range(1, steps + 1):
        t0 = time.perf_counter()
        o = rng.randint(0, len(tr) - T - 1, B)
        x = np.stack([tr[i:i + T] for i in o]); y = np.stack([tr[i + 1:i + T + 1] for i in o])
        loss, G = gpt_loss_grads(x, y, P)
        adam_step(P, G, state, LR); recent.append(loss)
        busy += time.perf_counter() - t0
        if step % CHECK_EVERY == 0:
            curve.append((step, np.mean(recent) / np.log(2) / cpt, bits_per_char(P, vx, vy,
                vocab))); recent = []
    return P, np.array(curve), busy / steps
```

- `ids[:n * T].reshape(n, T)` — the held-out ids cut into n windows of T; the targets are the same shifted by one.
- `sum(len(vocab[i]) for i in vy.ravel())` — how many characters the target tokens spell: one each for characters, more for BPE.
- `np.stack([tr[i:i + T] for i in o])` — B random windows stacked into one (B, T) array.
- `busy += time.perf_counter() - t0` — only the training step is timed; the held-out checks are not.

ngram_bits: the counting baseline

MATH

$$p(c \mid context) = (count(context, c) + k) / (count(context) + k\,V)$$

CODE

```
def ngram_bits(tr_text, va_text, order, vocab_size):
    def counts(text):
        c1, c2 = Counter(), Counter()
        for i in range(order - 1, len(text)):
            ctx = text[i - order + 1:i]; c1[ctx] += 1; c2[ctx, text[i]] += 1
        return c1, c2
    def bits(c1, c2, text, k):
        tot = 0.0
        for i in range(order - 1, len(text)):
            ctx = text[i - order + 1:i]
            tot -= np.log2((c2[ctx, text[i]] + k) / (c1[ctx] + k * vocab_size))
        return tot / (len(text) - order + 1)
    cut = int(0.9 * len(tr_text))
    c1, c2 = counts(tr_text[:cut])
    k = min((0.01, 0.1, 1.0), key=lambda k: bits(c1, c2, tr_text[cut:], k))
    c1, c2 = counts(tr_text)
    return bits(c1, c2, va_text, k), k, len(c1)
```

- `c2[ctx, text[i]] += 1` — a Counter keyed by a (context, character) tuple.
- `min((0.01, 0.1, 1.0), key=lambda k: bits(...))` — the smoothing with the lowest bits on the part of the training text held back.

sample: writing text

CODE

```
def sample(P, vocab, n, rng, start):
    ids = list(start)
    for _ in range(n):
        lg, _ = gpt_forward(np.array([ids[-T:]]), P)
        p = np.exp(lg[0, -1] - lg[0, -1].max()); p /= p.sum()
        ids.append(int(rng.choice(len(vocab), p=p)))
    return decode(ids[len(start):], vocab)
```

- `np.array([ids[-T:]])` — the last T tokens as a batch of one: the model cannot see further back.
- `rng.choice(len(vocab), p=p)` — draws one token with the model's probabilities (temperature 1).

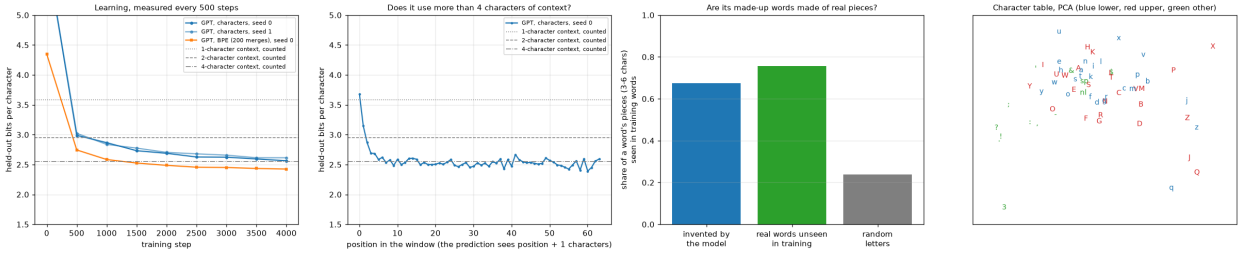
demo_samples: invented words and their pieces

CODE

```
known = set(re.findall(r"[a-z]+", tr_text.lower()))
gen = re.findall(r"[a-z]+", long.lower())
new = sorted({w for w in gen if w not in known and len(w) >= 4})
grams = {g for w in known for g in char_ngrams(w)}
piece_share = lambda ws: np.mean([np.mean([g in grams for g in char_ngrams(w)]) for w in ws])
letters = np.array(list("".join(re.findall(r"[a-z]", tr_text.lower()))))
rng = np.random.RandomState(2)
rand = ["".join(rng.choice(letters, len(w))) for w in new]
```

- `re.findall(r"[a-z]+", text)` — every run of letters: the words.
- `{g for w in known for g in char_ngrams(w)}` — Day 88's pieces of every training word, in one set.
- `rng.choice(letters, len(w))` — random letters drawn with the training text's letter frequencies.

REAL GRAPH



Real graph from the script: learning against the counting models, the loss by position in the window, invented words, and the character table.

Quick Syntax Glossary — new or reinforced constructs

- `np.broadcast_to(a, shape)` — a read-only view repeated to a larger shape, no copy.
- `d.setdefault(k, default)` — d[k], created as default the first time.
- `m[:] = ...` — write into an existing array instead of rebinding the name.
- `reversed(list(zip(a, b)))` — pairs, last first.
- `re.findall(pattern, s)` — every match of a regular expression.
- `rng.choice(n, p=p)` — one draw from a probability distribution.

End of Day 90 syntax reference. For the experiments and what they showed, see the Day 90 complete guide.